

Contents

Milestone verification — what actually gates it, and where it leaks	1
How a milestone reaches VERIFIED	1
Enforced vs judgment	1
Findings	2
The observed case	2
Recommended fixes, ranked	3
What is not broken	3

Milestone verification — what actually gates it, and where it leaks

*Findings from reading the code end to end, 2026-09-18. Scope: how a **group_milestones** row reaches VERIFIED, what is enforced, and what is left to the model.*

How a milestone reaches VERIFIED

The governing rule, from `helpers/milestone-transitions.ts`:

The agent may move a milestone anywhere that asserts no achievement. It may never move one past evidence.

Progress	Agent	Why
NOT_STARTED	allowed	asserts nothing
IN_PROGRESS	allowed	a claim about activity; a wrong one costs a correction the team can see
SUBMITTED	never	means <i>the team says this is done</i> — their claim, not the agent’s
VERIFIED	only with a citable result	a claim about achievement, and a grade rests on it

Evidence is required for **any** progress change, and may be `repo_file`, `attachment`, `artifact`, `message` or `result`. For VERIFIED the transitions table refuses the agent outright, and `screenAgentVerification` narrowly overrides that refusal when the evidence cites a result that is (1) present, (2) owned by this team, (3) VERIFIED_POSITIVE|NEGATIVE, and (4) backed by an attachment (`isCitableResult`).

A `repo_file` item is never sufficient on its own: “*evidence that an agent read a file is still not evidence that the file says what it thought.*” So a verified milestone always carries a `result` item as the key, and usually repo permalinks as the argument.

The only other route is a staff member setting VERIFIED directly — a human taking responsibility.

Enforced vs judgment

Question	Answered by
Does a verified result exist, owned by this team, with an artifact?	code (<code>screenAgentVerification</code>)
Is that result <i>about this milestone</i> ?	the model — nothing checks
Which milestone did the verdict cover?	nobody — not stored anywhere
Did the repo evidence actually support the claim?	the model

Findings

F1 — one result can verify any number of milestones. The check is `resultItems.some(...)`: “does the evidence cite at least one citable result”. It never reads the milestone. Nothing marks a result as consumed, and a citable result stays citable indefinitely — so the same id can unlock a milestone this turn and another one next month. The per-turn cap of 3 milestone writes is a budget, not a semantic limit.

F2 — no record of which milestone a verdict was for. `guide_verifications` holds `result_id`, `group_id`, `verdict`, `summary`, `checks`, `metrics`, `message_id`. There is no milestone column. “Verified for milestone 3” is an inference the model makes in prose and the platform never stores.

F3 — the authorising evidence is the only one you cannot inspect. `evidenceNavTarget` resolves `attachment`, `artifact` and `message`; `repo_file` renders as a GitHub permalink (path + commit sha + line range). `result` falls through to `default: return null`, so `EvidenceRow` draws it as plain, unclickable text. A mentor auditing a verified milestone sees clickable repo links — which legally could not have unlocked it — and a grey sentence where the actual key was. The `result_id` is in the database and nowhere in the product.

F4 — agent-written labels hide reuse. `label` is written by whoever supplied the evidence, by design (“so it can say ‘the training loop, which sets no seed’ rather than ‘train.py’”). So one result can appear under four milestones wearing four different descriptions, and nothing on screen reveals they are the same row.

F5 — a refused turn does not roll back its writes. `completeTurn` withdraws *artifacts* from a withheld turn. Milestone moves and verifications went through the internal routes mid-loop, before any reviewer decided, and are never undone. So a turn the outbound reviewer kills still advances project state — the student sees a refusal sentence while their milestone panel changes underneath them.

F6 — terminal statuses strand results. Status may only move *from* `PENDING`. A result filed with numbers and no file lands `UNVERIFIED` and can never be promoted. A `NOT_VERIFIED` or `INCONSISTENT` verdict is final, and because “Open results” filters on `PENDING|UNVERIFIED`, the row also disappears from the agent’s context. The natural recovery — agent says “no log shows this run”, student uploads the log — does not work: there is nothing left to re-verify, and nothing says so.

F7 — the outbound reviewer cannot see any of it. It receives the brief, the thread, the question, the answer and the extracted numbers. Not which writes applied or were refused. So an answer claiming “milestone 1 is now verified” when that write was refused passes unchallenged.

The observed case

Group `iQZjwFNPr-tn4tzWNJYoU`, prod, 2026-09-15. Four milestones show as passed, each citing a `result` item describing the same preprocessing logs:

#	Result evidence label
1	“Preprocessing logs showing 11,455 PV and 26,337 Cassava images processed”
2	“Preprocessing logs showing 11,455 PV images processed”
3	“Preprocessing logs showing 26,337 Cassava images processed”
4	“Preprocessing logs for all 4 levels.”

Four sentences, one underlying artifact (F1 + F4). The brief at that moment listed exactly one citable result, so these moves must have cited it.

11,455 and 26,337 are also the numbers the outbound reviewer refused the answer over — read off an uploaded screenshot, untraceable because images contribute nothing to `read_text`. So the turn was withheld and the milestones moved anyway (F5).

Not yet confirmed: whether all four cite literally the same `result_id`. Settle it with `SELECT position, title, evidence FROM group_milestones WHERE group_id = '...' AND progress = 'VERIFIED'` and count distinct result ids.

Recommended fixes, ranked

1. **Record intent at verification time.** Let `record_verification` name the milestone(s) a verdict covers (a list — one report may legitimately evidence several), store it on `guide_verifications`, and have `screenAgentVerification` require the cited verification to name *this* milestone. Turns the claim into a stored fact a mentor can audit, without forbidding legitimate multi-milestone coverage. Fixes F1 and F2.
2. **Make the result evidence clickable.** Give `result` a nav target in `evidenceNavTarget` — the Results tab already lists results by id. One small change, and reuse becomes self-policing: four milestones pointing at one report would be visible at a glance. Fixes F3, mitigates F4.
3. **Give the reviewer the turn’s write outcomes.** Already built on the `fix/outbound-reviewer-evidence` branch — the evidence list gains `write applied / write REFUSED this turn`. Fixes F7 and makes F5 visible for the first time.
4. **Decide what a withheld turn should do to state.** Either roll back milestone moves like artifacts, or tell the student plainly that the record changed even though the answer did not arrive. Currently neither happens. Addresses F5.
5. **Give terminal results a way forward.** Allow a re-submission to supersede a `NOT_VERIFIED` row, or say in the refusal that a new submission is the only remedy. Addresses F6.

What is not broken

`VERIFIED_POSITIVE` is the happy path and works as designed. The corroboration rule does the job it was built for — it stops a verification claim with nothing behind it. A strict one-result-per-milestone rule would be wrong: a single week-1 report genuinely evidences both “read & digest” and a defined protocol. The gap is not that reuse is possible; it is that nothing distinguishes justified reuse from a student talking the agent into it, and nothing records which milestone a verdict was actually about.